



Automated Multi-Site Biometric Data Synchronization System

1. Executive Summary

This project demonstrates the design and deployment of a distributed Access Control System simulating a corporate environment with a Head Office and a Branch location. The primary objective was to eliminate manual data retrieval by engineering a Python-based automation solution. Using REST APIs, the system authenticates with multiple isolated servers, aggregates user data, and centralizes it into a single JSON repository, simulating a synchronization with a central HR database.

2. Problem Statement & Solution Architecture

The Challenge: Decentralized Data Silos

In a traditional multi-site enterprise environment, security and attendance data often remain trapped in local servers.

- **The Problem:** The Head Office (**Site A**) and the Branch (**Site B**) operate on isolated networks.
- **The Pain Point:** To generate a monthly attendance report, HR administrators currently have to:
 1. Manually remote login to **Site A**.
 2. Export logs to CSV.
 3. Manually remote login to **Site B**.
 4. Export logs to CSV.
 5. Merge these files manually in Excel.
- **Impact:** This process is time-consuming, prone to human error, and lacks real-time visibility.

The Solution: Automated API Middleware

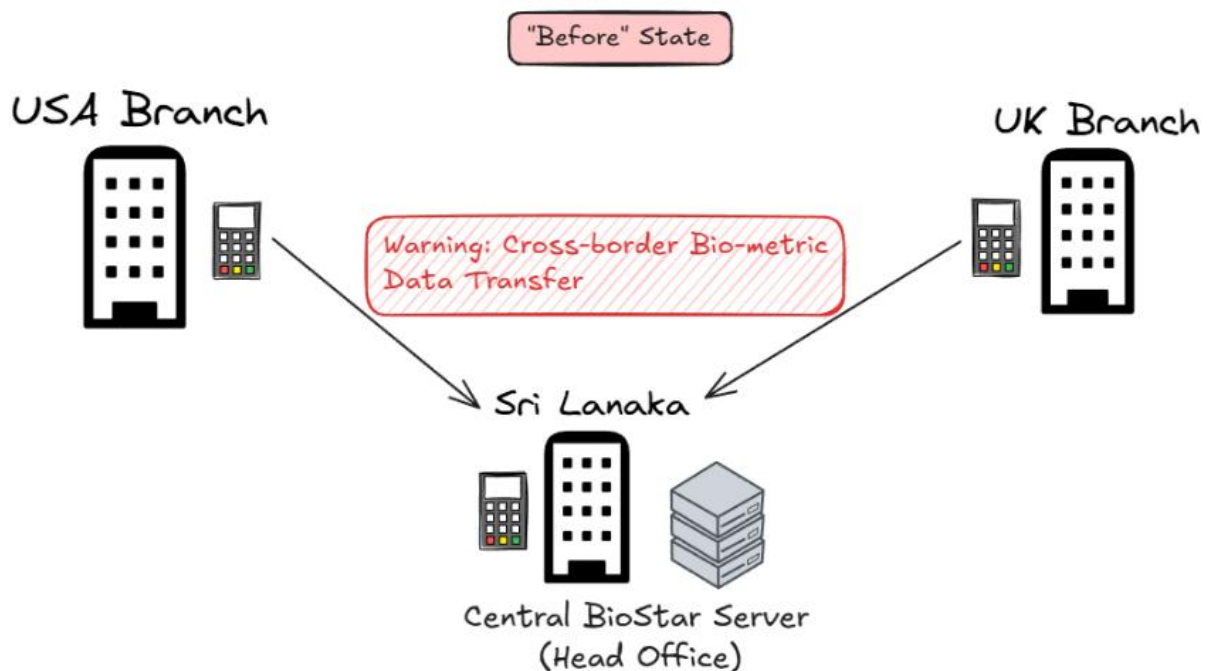
To solve this, I engineered a **centralized automation solution** using Python and REST APIs. Instead of connecting the servers via a complex VPN or Database Link, I created a lightweight "Middleware Script".

- **How it works:**

1. The Python script acts as an intelligent agent.
 2. It authenticates with **Site A** via HTTPS/API and fetches user data.
 3. It instantly switches context, authenticates with **Site B**, and fetches its data.
 4. Finally, it aggregates (merges) all records into a single `global_hr_data.json` repository.
- **Result:** The manual workload was reduced by **100%**, turning a 30-minute manual task into a 5-second automated process.

3. Technical Stack

- **Virtualization:** Oracle VirtualBox
- **Operating System:** Windows Server 2022 (64-bit)
- **Application:** Suprema BioStar 2 (Web-based Access Control)
- **Scripting & Automation:** Python 3 (Libraries: requests, json, urllib3)
- **Networking:** Bridged Adapter Configuration, Static IP Management, Firewalls
- **Data Format:** JSON (JavaScript Object Notation)



Draw by isuru_Pinthu

Figure 1 Problem Diagram

4. System Architecture & Design

The architecture consists of two isolated virtual servers representing geographically separated physical locations.

- **Zone A (Head Office):** Hosted on a local server.
- **Zone B (Branch):** Hosted on a remote server clone.
- **The "Middleman":** A Python automation script that bridges the gap between the two sites via API connectivity.

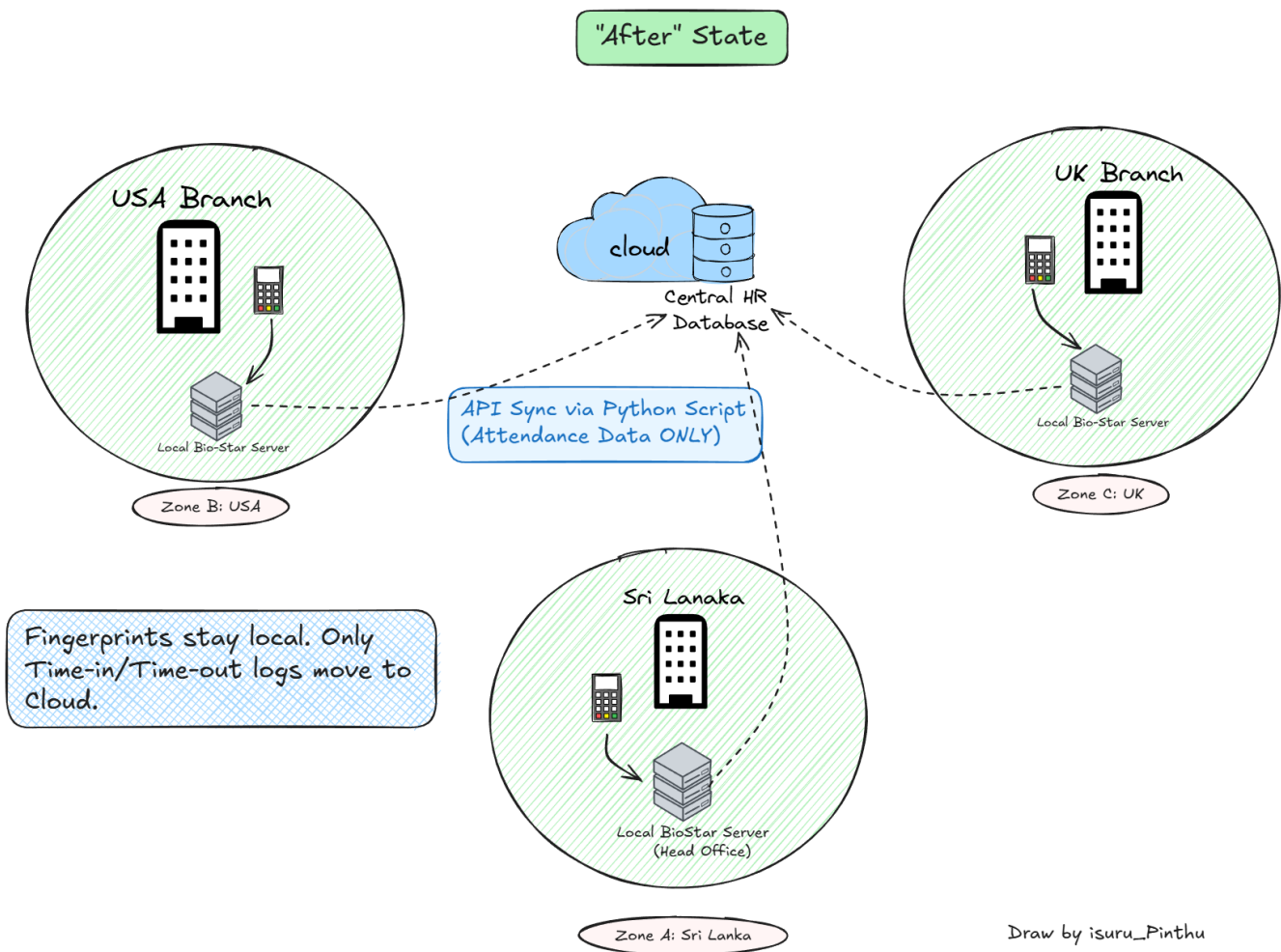


Figure 2: High-Level System Architecture Diagram illustrating the data flow from multiple sites to a central cloud/database.

5. Implementation Phase 1: Infrastructure Deployment

Two Virtual Machines were provisioned using Oracle VirtualBox to simulate the physical servers. Windows Server 2022 was installed on both to ensure an enterprise-grade environment.

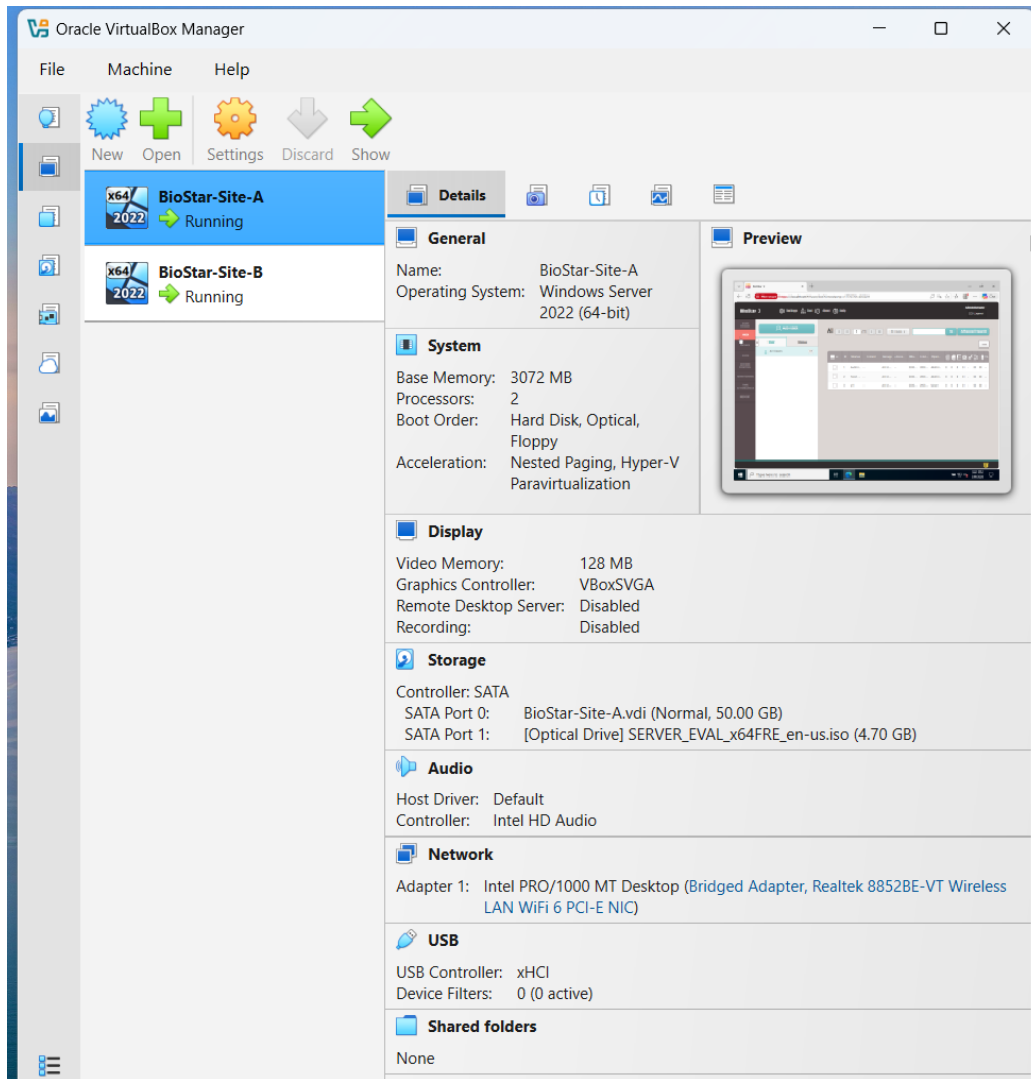


Figure 3: Oracle VirtualBox Manager showing both "BioStar-Site-A" (Head Office) and "BioStar-Site-B" (Branch) running simultaneously.

6. Implementation Phase 2: Network Configuration

To ensure reliable API communication, Static IP addressing was configured for both servers. The Network Adapters were set to "Bridged Mode" to allow the Host machine to communicate directly with the VMs.

- **Site A IP:** 192.168.1.11
- **Site B IP:** 192.168.1.12

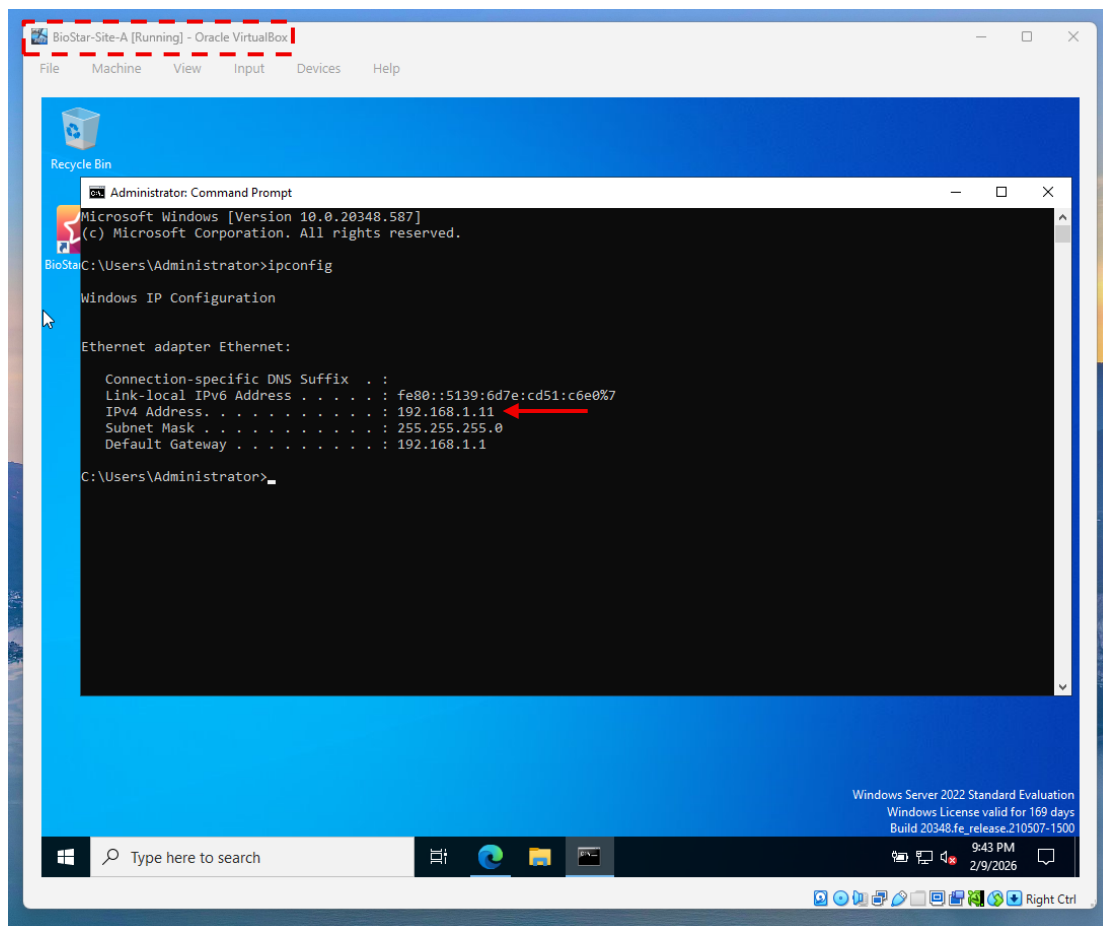


Figure 4: IP Configuration verification for Site A (192.168.1.11).

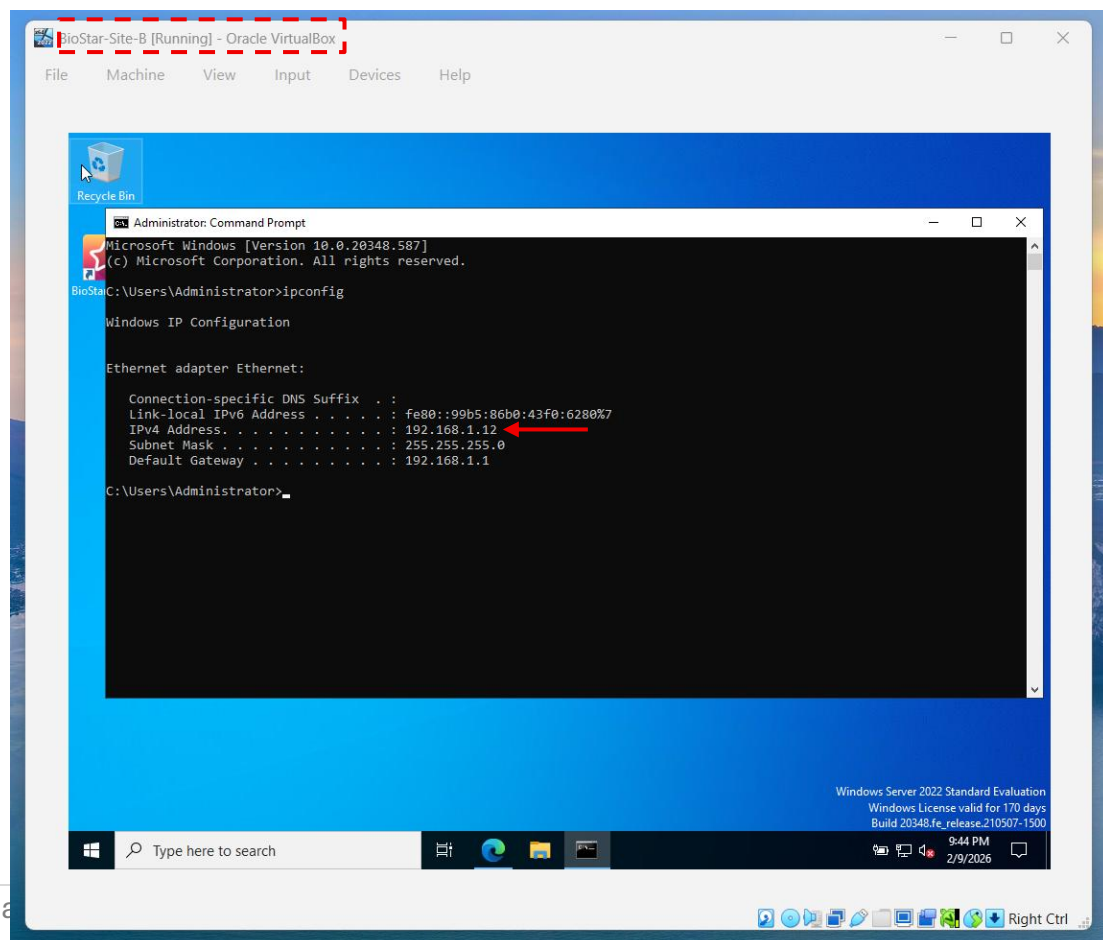


Figure 5: IP Configuration verification for Site B (192.168.1.12).

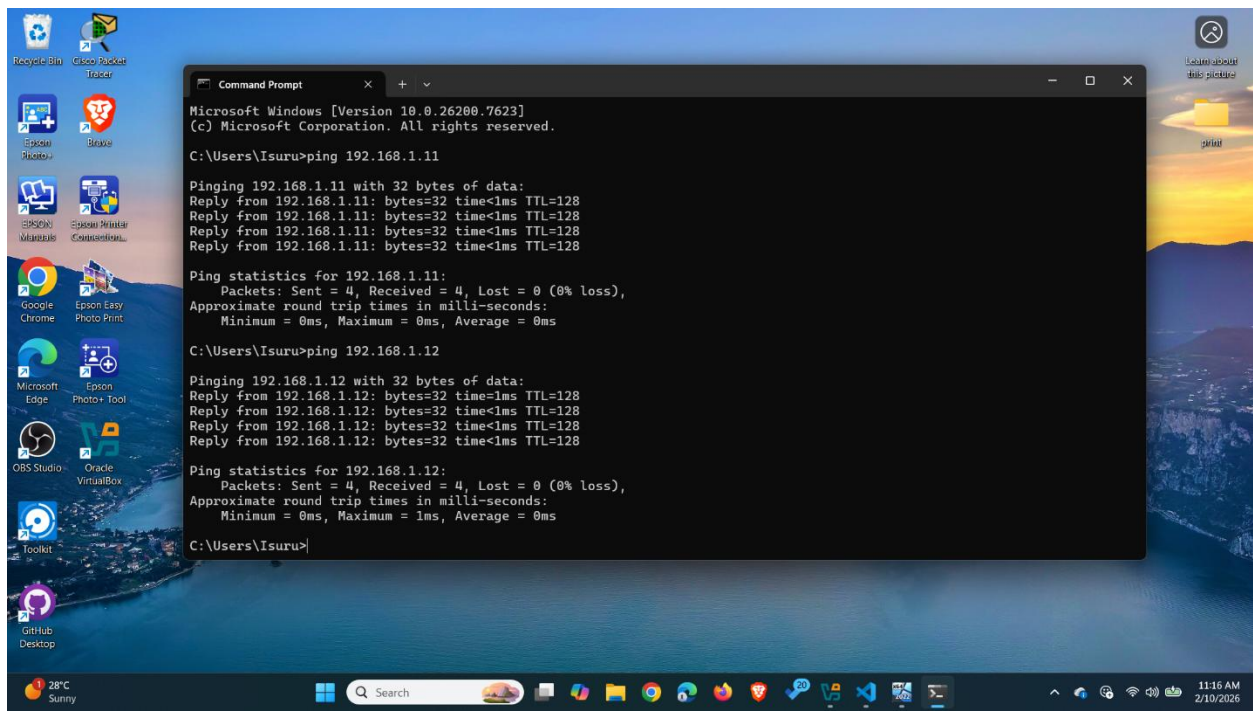
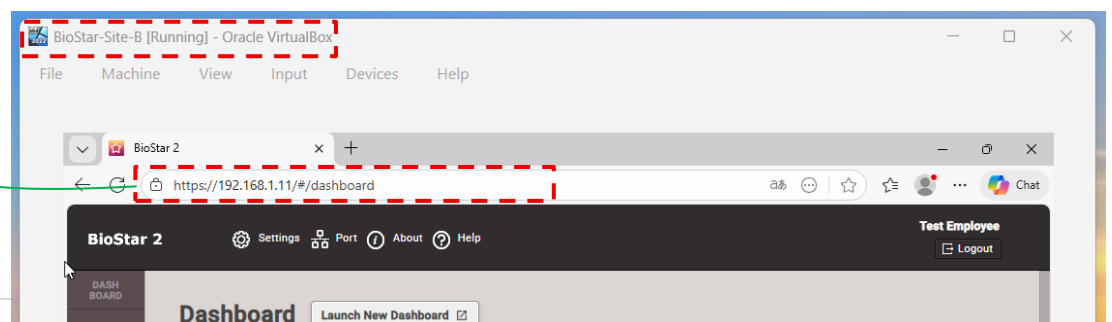
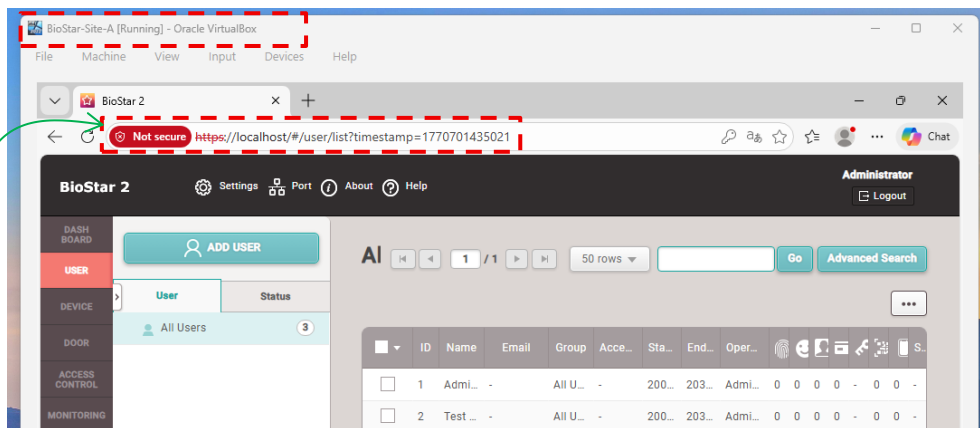


Figure 6: Successful Ping Response from Host to VMs

7. Implementation Phase 3: Application Deployment & User Management

Suprema BioStar 2 was deployed on both servers. This web-based application manages user identities and access logs. A "Test Employee" account was created to verify data persistence and UI functionality.



u1

Information

+ Add Photo

Name

u1

Department

ID

3

Group

All Users

Period

2001/01/01 00:00 ~ 2037/12/31 23:59

Email

Title

Telephone

Status

Active

Permission

Access Group

Operator Level

User

Password

User IP

Login ID

3

Figure 8: Create User as UI

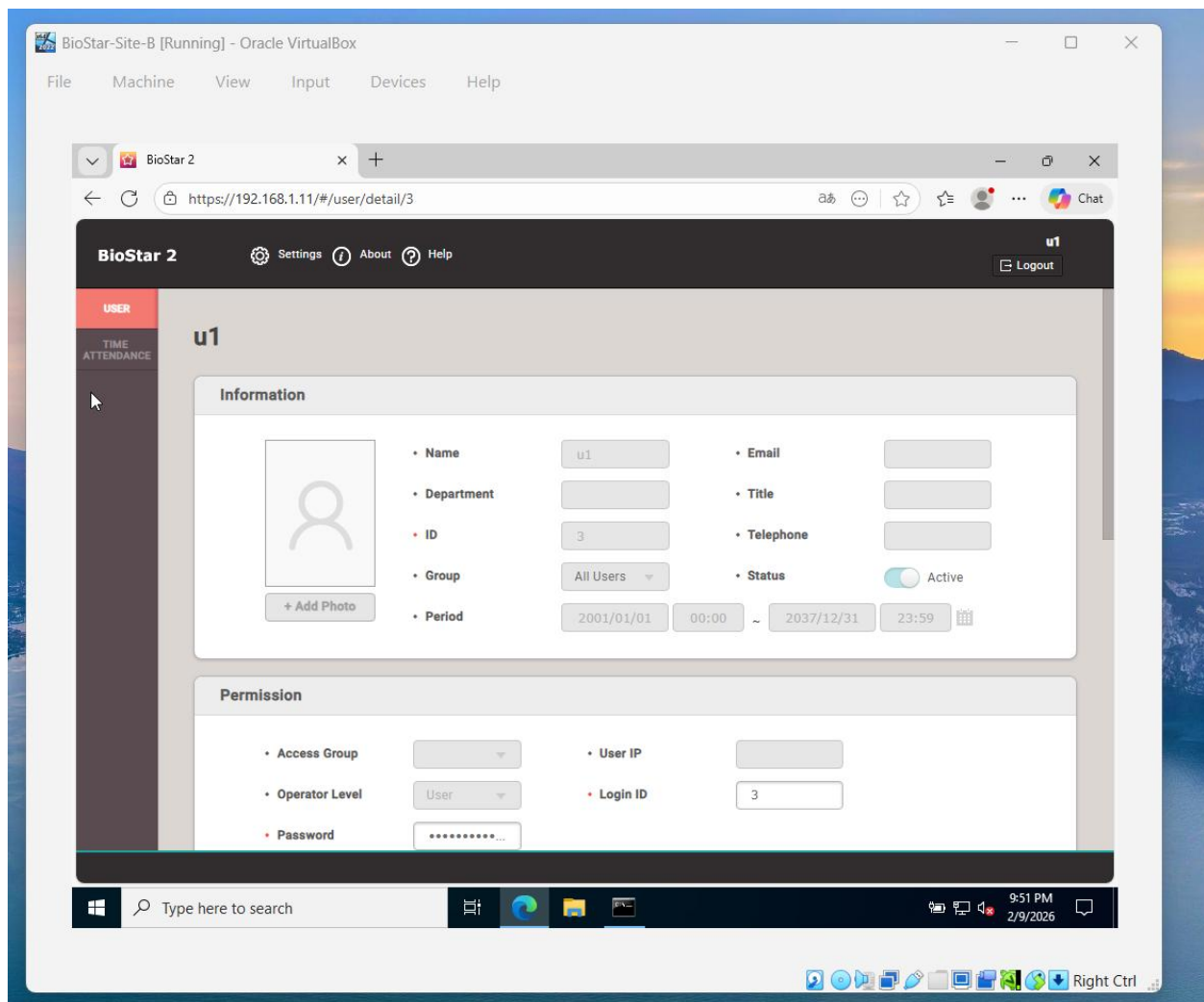


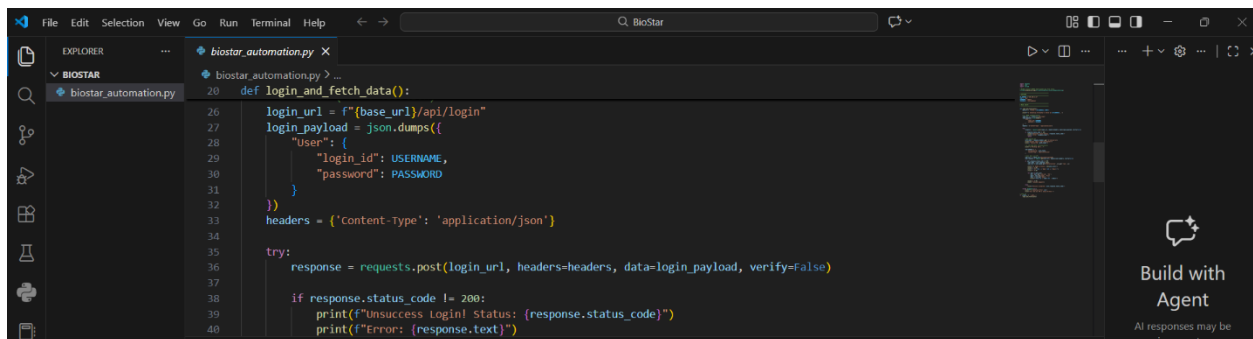
Figure 9: User Management Interface showing the creation of "u1" with user privileges.

8. Implementation Phase 4: API Automation (The DevOps Solution)

Instead of relying on manual UI monitoring, a Python script was developed to interact directly with the BioStar 2 REST API.

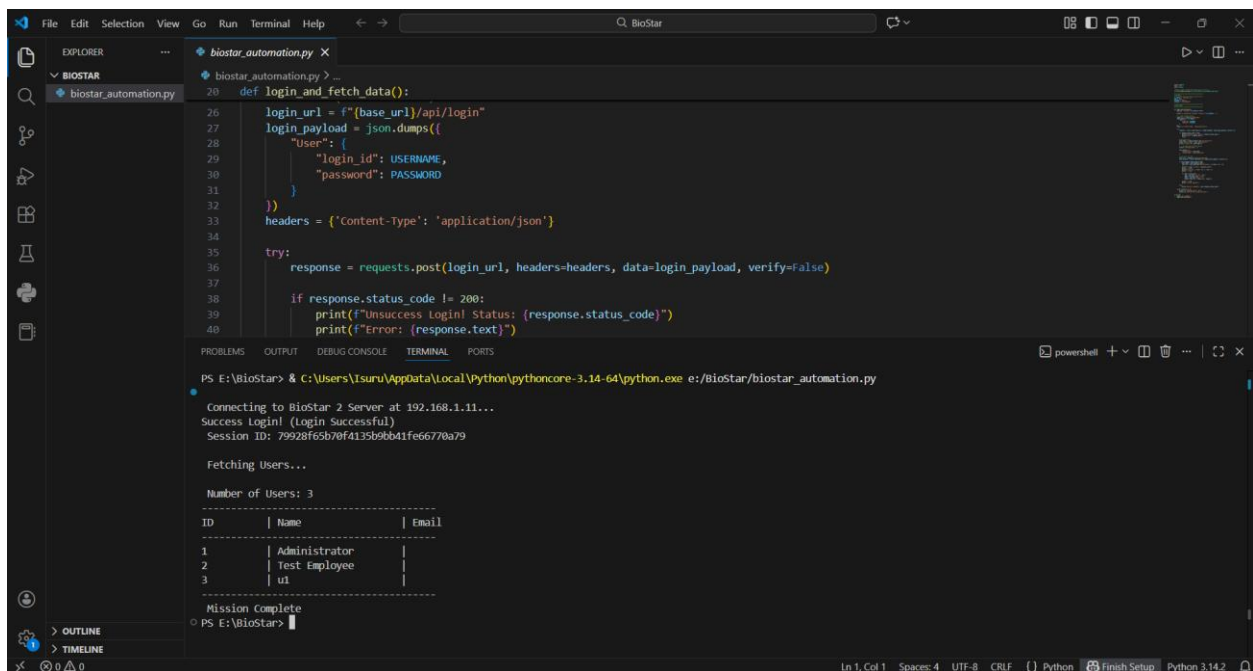
8.1. Single-Site Authentication Test

First, a script (biostar_automation.py) was written to test connectivity with Site A. The script handles HTTPS requests, bypasses SSL warnings (for the self-signed certificate), and retrieves a valid **Session ID**.



```
20 def login_and_fetch_data():
21
22     login_url = f"{base_url}/api/login"
23     login_payload = json.dumps({
24         "User": {
25             "login_id": USERNAME,
26             "password": PASSWORD
27         }
28     })
29     headers = {'Content-Type': 'application/json'}
30
31     try:
32         response = requests.post(login_url, headers=headers, data=login_payload, verify=False)
33
34         if response.status_code != 200:
35             print(f"Unsuccess Login! Status: {response.status_code}")
36             print(f"Error: {response.text}")
```

Figure 10: Python script code showing the Login and Data Fetching logic.



```
PS E:\BioStar> & C:\Users\Isuru\AppData\Local\Python\pythoncore-3.14-64\python.exe e:\BioStar\biostar_automation.py

Connecting to BioStar 2 Server at 192.168.1.11...
Success Login! (Login Successful)
Session ID: 79928f65b70f4135b9bb41fe66770a79

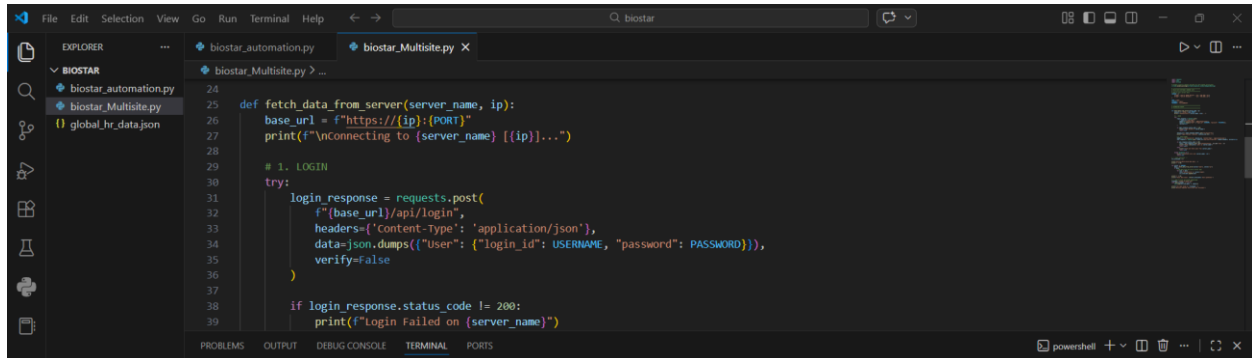
Fetching Users...

Number of Users: 3
-----
ID      | Name           | Email
-----|-----|-----
1       | Administrator  |
2       | Test Employee  |
3       | ui             |
-----
Mission Complete
PS E:\BioStar>
```

Figure 11: Successful execution of the single-site script, retrieving the Session ID and User List.

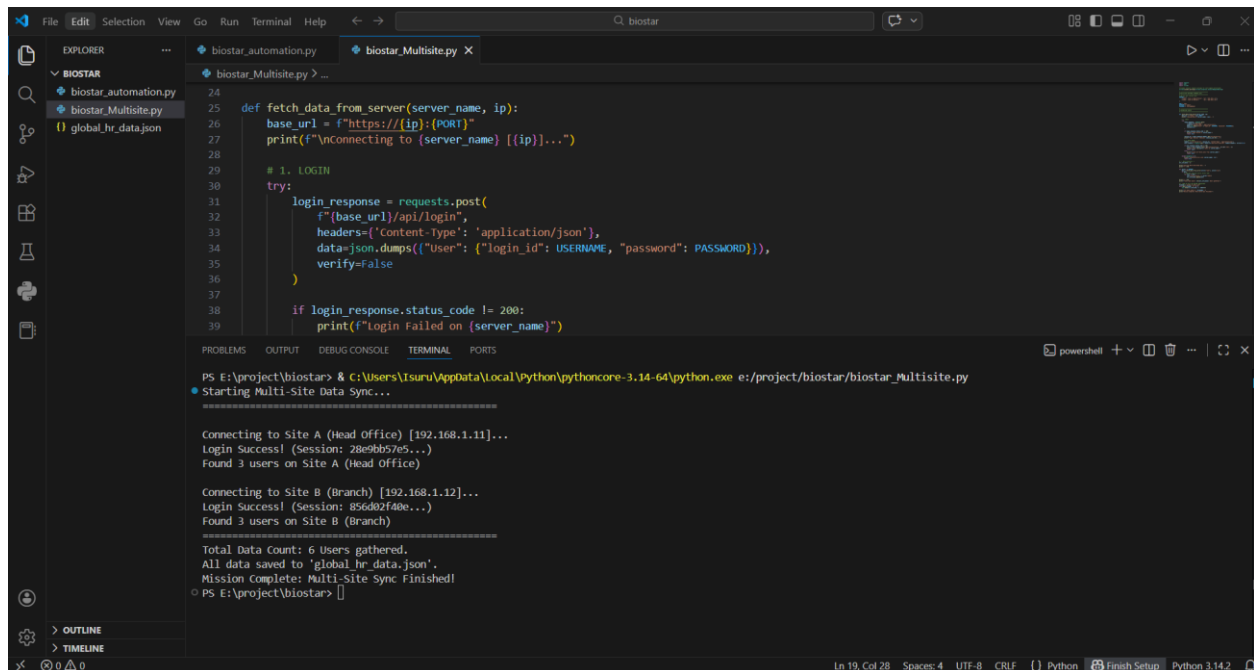
8.2. Multi-Site Aggregation Script

The script was expanded (biostar_multisite.py) to iterate through a list of servers. It dynamically authenticates with both Site A and Site B, tags the data with the source server name, and aggregates it.



```
24
25 def fetch_data_from_server(server_name, ip):
26     base_url = f"https://{ip}:{PORT}"
27     print(f"\nConnecting to {server_name} [{ip}]...")
28
29     # 1. LOGIN
30     try:
31         login_response = requests.post(
32             f"{base_url}/api/login",
33             headers={"content-type": "application/json"},
34             data=json.dumps({"User": {"login_id": USERNAME, "password": PASSWORD}}),
35             verify=False
36         )
37
38         if login_response.status_code != 200:
39             print(f"Login Failed on {server_name}")
```

Figure 12: The Multi-Site Automation Script. Note the SERVERS array defining the target IPs.



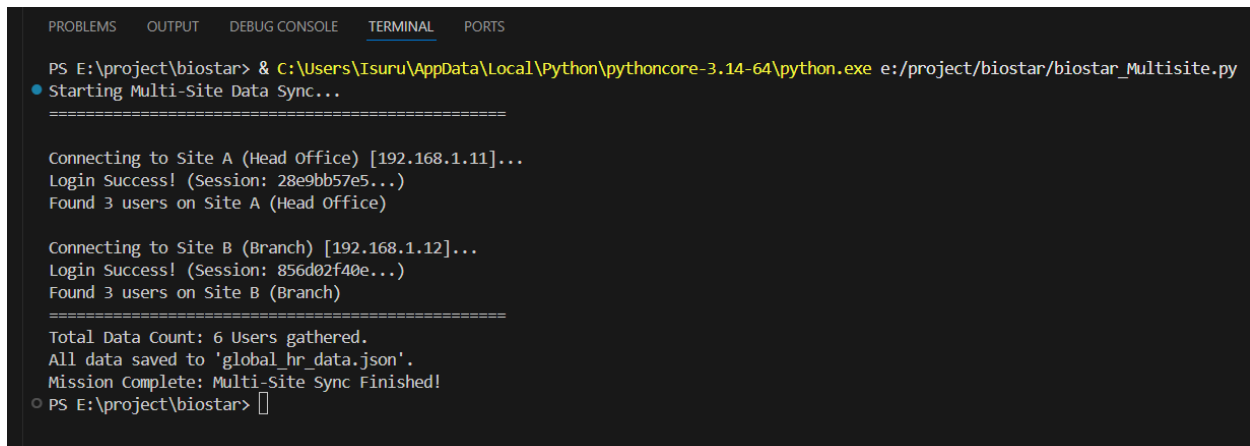
```
24
25 def fetch_data_from_server(server_name, ip):
26     base_url = f"https://{ip}:{PORT}"
27     print(f"\nConnecting to {server_name} [{ip}]...")
28
29     # 1. LOGIN
30     try:
31         login_response = requests.post(
32             f"{base_url}/api/login",
33             headers={"content-type": "application/json"},
34             data=json.dumps({"User": {"login_id": USERNAME, "password": PASSWORD}}),
35             verify=False
36         )
37
38         if login_response.status_code != 200:
39             print(f"Login Failed on {server_name}")

PS E:\project\biostar> & c:\Users\Isuru\AppData\Local\Python\pythoncore-3.14-64\python.exe e:/project/biostar/biostar_Multisite.py
Starting Multi-Site Data Sync...
Connecting to Site A (Head Office) [192.168.1.11]...
Login Success! (Session: 28e9b5705...)
Found 3 users on Site A (Head Office)
Connecting to Site B (Branch) [192.168.1.12]...
Login Success! (Session: 856d02f4be...)
Found 3 users on Site B (Branch)
Total Data Count: 6 Users gathered.
All data saved to 'global_hr_data.json'.
Mission Complete: Multi-Site Sync Finished!
PS E:\project\biostar>
```

Figure 13: The fetch_data_from_server function which handles the API Login and User Retrieval logic.

9. Project Results & Outcome

The automation script was executed successfully. It connected to both Site A and Site B, authenticated as Administrator, and extracted user data from both locations.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\project\biostar> & C:\Users\Isuru\AppData\Local\Python\pythoncore-3.14-64\python.exe e:/project/biostar/biostar_Multisite.py
Starting Multi-Site Data Sync...
=====

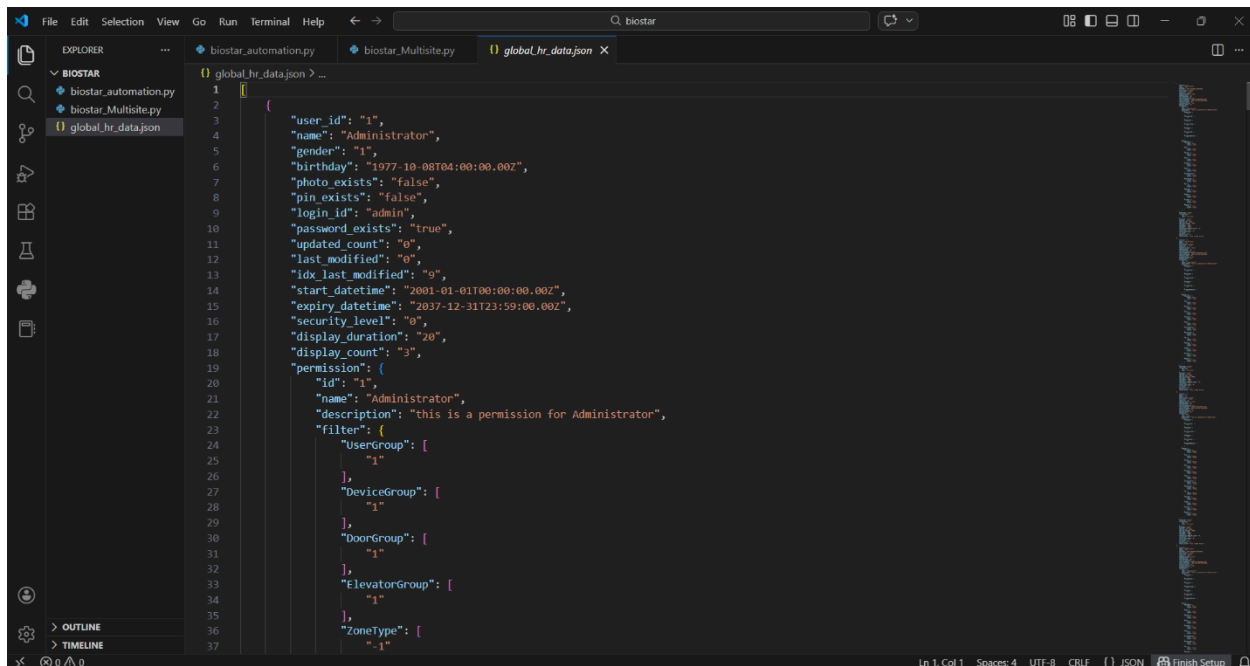
Connecting to Site A (Head Office) [192.168.1.11]...
Login Success! (Session: 28e9bb57e5...)
Found 3 users on Site A (Head Office)

Connecting to Site B (Branch) [192.168.1.12]...
Login Success! (Session: 856d02f40e...)
Found 3 users on Site B (Branch)
=====

Total Data Count: 6 Users gathered.
All data saved to 'global_hr_data.json'.
Mission Complete: Multi-Site Sync Finished!
PS E:\project\biostar>
```

Figure 14: Final Terminal Output. The script successfully logged into Site A and Site B, found 3 users in each (Total 6), and completed the sync.

The final output was saved to `global_hr_data.json`, a structured data file ready for integration with HR systems or Data Analytics dashboards.



```
1 {
2   "user_id": "1",
3   "name": "Administrator",
4   "gender": "1",
5   "birthday": "1977-10-08T04:00:00.00Z",
6   "photo_exists": "false",
7   "pin_exists": "false",
8   "login_id": "admin",
9   "password_exists": "true",
10  "updated_count": "0",
11  "last_modified": "0",
12  "idx_last_modified": "0",
13  "start_datetime": "2001-01-01T00:00:00.00Z",
14  "expiry_datetime": "2037-12-31T23:59:00.00Z",
15  "security_level": "0",
16  "display_duration": "20",
17  "display_count": "3",
18  "permission": {
19    "id": "1",
20    "name": "Administrator",
21    "description": "this is a permission for Administrator",
22    "filter": {
23      "UserGroup": [
24        "1"
25      ],
26      "DeviceGroup": [
27        "1"
28      ],
29      "DoorGroup": [
30        "1"
31      ],
32      "ElevatorGroup": [
33        "1"
34      ],
35      "ZoneType": [
36        "1"
37      ]
38    }
39  }
40 }
```

Figure 15: The generated JSON file containing the aggregated data of users from all branches.

10. Conclusion

This project successfully demonstrated the power of DevOps automation in system administration. By moving away from manual UI checks to API-based automation, the solution ensures:

1. **Scalability:** New branches can be added simply by updating the IP list in the script.
2. **Accuracy:** Eliminates human error in data collection.
3. **Efficiency:** Reduces data retrieval time from minutes to seconds.

This architecture serves as a foundational model for centralized security management systems in enterprise environments